

End-to-end generation without writing the app by hand

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 5 of 12 · English notes

Session 5 finished the task-management stack that session 4 had paused mid-frontend: resume the same conversation so the orchestrator continues the to-do list, run backend and frontend, exercise Swagger and the Kanban UI, then extend the repo with Docker/CI-shaped skills — still without the trainer typing application code. “Without writing the app by hand” does not mean “without engineering judgement.” It means the *source of the handlers and components is the agent loop*, steered by CLAUDE.md, skills, OpenAPI, Figma/MCP, and permission/plan modes.

YOUR GENERATED UI WILL NOT MATCH THE RECORDING

Ports (3000 / 5173), Vite, Tailwind, Playwright, and Docker Desktop prompts are demo-time. If Claude opens Docker Desktop uninvited, tighten the skill. Prefer current **Claude Code** behaviour over this walkthrough’s exact clicks.

Resume is a feature, not a new prompt

Because the previous session stopped during verification, the trainer stayed in the *same conversation* and asked to continue code generation. The agentic loop already had a to-do list; remaining items (pages, routers, wiring) picked up. That only works if context is still there — a new session would need you to re-state artefacts (OpenAPI path, Figma URL, skills). Permission mode still applies to each remaining tool call.

Run what was generated

A short prompt — “run my backend and frontend application” — is enough if `package.json` scripts exist; the agent reads them instead of you pasting npm incantations. Document start/stop in README so Claude does not invent commands. In the lab: API docs on localhost:3000, SPA on 5173. Register/login, create a task (“learn Claude”), move columns. Missing drag-and-drop was a missing requirement, not a model failure — you can add it in a follow-up with a screenshot plus the route name to save tokens.

- Swagger: set the server to local, obtain a bearer, call locked routes such as current user and tasks.
- Hot reload: if the frontend is in a Vite-style dev server, CLAUDE.md-guided edits (filters from three rows to one) appear without a full rebuild.
- Ignore or skip verification steps when they dominate a small project (auth rate-limit theatre); say so explicitly.
- Humanizer words (“cooking,” “marinating”) in the UI are loaders, not extra billed vocabulary and not a separate model.

Homework before generation beats giant prompts

Pedro asked whether per-folder instructions fight each other in multi-layer solutions. Vijay: keep a global clarify-on-conflict rule; put naming conventions and design patterns in the layer that owns them (Java camelCase in backend CLAUDE.md, CSS rules under a static `public/` note). Templates for controllers (inject ILogger, mediator, ...) are good if you are precise. Over-specifying every GoF pattern explodes iterations and cost — the cohort’s files were intentionally not that deep.

Container and pipeline skills: still no hand-written app

Once the app ran, the trainer generated Dockerfiles via skills (nginx-style frontend image, backend image with env/db notes), talked about a **mono repo** holding frontend, backend, Terraform, Kubernetes, Jenkins or GitHub Actions — each with its own skill. Ask Claude to write the skill file if you want, then slash it with a tight task (“frontend container only”). Verify `.gitignore` so `node_modules` never lands on GitHub. That is still end-to-end delivery work done by agents, not by you authoring JavaScript.

What “production-ready” meant in the room

The slogan from earlier sessions — production-ready apps without a line of code — is a classroom claim about *who types the implementation*. You still review diffs, secrets, OpenAPI drift, accessibility, and whether Playwright versus Selenium belongs on the plan. Tests the agent runs during verify may not even be files you keep; the trainer caught validation without a test module and noted you should capture those cases as real tests so you do not pay to regenerate them later.

DO NOT CONFUSE EXAM CODING WITH THIS LAB

Foundation written scenarios are not “open the IDE and generate the app.” They ask you to reason about plan vs execute, CLAUDE.md vs skills, MCP vs attachments, and token-aware attachments. This lab is how those pieces feel when assembled.

Bridge to hooks (next module)

The session closed by naming the next Claude Code concepts: **sub-agents** (already introduced), **hooks**, and **plugins**. End-to-end generation that formats code by asking the model to “follow Prettier in CLAUDE.md” is the expensive path. Module 06 moves that work to local post-tool hooks so the same greenfield app costs hundreds to thousands fewer tokens on reprocessing.

Official documentation

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

Running, resuming, and skill-driven generation in the coding client.

- **Claude Docs home**

<https://docs.claude.com/en/home>

Product behaviour if the IDE browser preview or Docker integration moved.

- **Rate limits**

<https://docs.claude.com/en/api/rate-limits>

Long verify loops and usage meters are why you skip redundant checks.

- **Anthropic Academy (Skilljar)**

<https://anthropic.skilljar.com>

Official courses; this lab is UST practice, not a Skilljar exercise file.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.